

AUTOMATA LANGUAGE AND COMPUTATION

UNIT-II

Regular Sets and Context Free Grammars: Properties of regular sets, Context-Free Grammars-derivation trees, Chomsky Normal Forms and Greibach Normal Forms, ambiguous and unambiguous grammars; minimization of finite automata.

1. REGULAR SETS

Regular languages can also be defined, from the empty set and from some finite number of singleton sets, by the operations of union, composition, and Kleene closure. Specifically, consider any alphabet. Then a regular set over is defined in the following way.

- The empty set $\emptyset$, the set $\{\epsilon\}$ containing only the empty string, and the set $\{a\}$ for each symbol a in Σ , are regular sets.
- If L_1 and L_2 are regular sets, then so are the union $L_1 \cup L_2$, the composition $L_1 L_2$, and the Kleene closure L_1^* .
- No other set is regular.

1.1 The Pumping Lemma for Regular Sets

Applications:

- The pumping lemma is a powerful tool for providing and proving certain language is regular or not.
- It is also useful in the development of algorithms to answer certain questions concerning finite automata, such as whether the language accepted by a given FA is finite or infinite.

Statement:

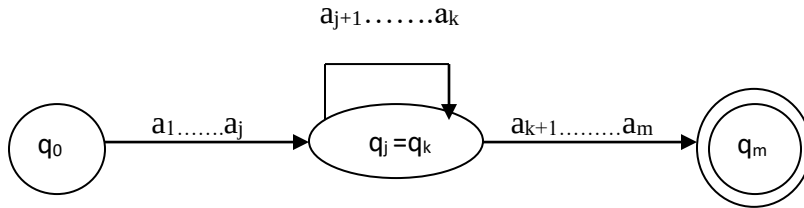
Let “ L ” be a regular set. Then there is a constant n such that if z is any word in L , and $|z| \geq n$. We may write $z=uvw$ in such a way that $|uv| \leq n$, $|v| \geq 1$ and for all $i \geq 0$, then $uv^i w$ is in L .

Proof:

If a language is accepted by a DFA $M = (Q, \Sigma, \delta, q_0, F)$ then it is regular with some particular number of states, say n . Consider a input of n or more symbols $a_1 a_2 \dots a_m$, $m \geq n$ and for $i=1, 2, \dots, m$.

Let $\delta(q_0, a_1 a_2 \dots a_i) = q_i$

It is not possible for each of the $n+1$ states $q_0, q_1, \dots, q_n$ to be distinct, since there are only different states. Thus there are two integers j and k , $0 \leq j < k \leq n$, such that $q_j = q_k$. The path labeled $a_1 a_2 \dots a_m$ in the transition diagram of M . Since $j < k$, the string $a_{j+1} \dots a_k$ is of length ≥ 1 .



If q_m is in F , that is $a_1 a_2 \dots a_m$ is in $L(n)$, then $a_1 a_2 \dots a_j a_{k+1} a_{k+2} \dots a_m$ is also in $L(n)$.

$$\begin{aligned} \delta(q_0, a_1 \dots a_j a_{k+1} \dots a_m) &= \delta(\delta(q_0, a_1 \dots a_j), a_{k+1} \dots a_m) \\ &= \delta(q_j, a_{k+1} \dots a_m) && (\text{since } \delta(q_0, a_1 \dots a_j) = q_j) \\ &= \delta(q_k, a_{k+1} \dots a_m) && (\text{since } q_j = q_k) \\ &= q_m \end{aligned}$$

which is the final state of the automata to be accepted.

Hence proved.

1.2 Properties of Regular Sets

Property 1: The regular sets are closed under union, concatenation, and Kleene closure.

Proof: Let Σ be a finite set of symbols and let L, L_1 and L_2 be sets of strings from Σ^* . The concatenation of L_1 and L_2 , denoted $L_1 L_2$, is the set $\{xy \mid x \text{ is in } L_1 \text{ and } y \text{ is in } L_2\}$. That is, the strings in $L_1 L_2$ are formed by choosing a string L_1 and following it by a string in L_2 , in all possible combinations. Define $L^0 = \{\epsilon\}$ and $L^i = L L^{i-1}$ for $i \geq 1$. The Kleene closure of L denoted L^* , is the set

$$L^* = \sum_{i=0}^{\infty} L^i$$

and the positive closure of L , denoted L^+ , is the set

$$L^+ = \sum_{i=1}^{\infty} L^i$$

That is, L^* denotes words constructed by concatenating any number of words from L . L^+ is the same, but the case of zero words, whose concatenation is defined to be ϵ , is excluded. Note that L^+ contains ϵ if and only if L does.

Property 2: The class of regular sets is closed under complementation. That is, if L is a regular set and $L \in \Sigma^*$, and then $\Sigma^* - L$ is a regular set.

Proof: Let L be $L(M)$ for DFA $M = (Q, \Sigma_1, \delta, q_0, F)$ and $L \in \Sigma^*$. First, we may assume $\Sigma_1 = \Sigma$, for if there are symbols in Σ_1 not in Σ , we may delete all transitions of M on symbols not in Σ . The fact

that $L \in \Sigma^*$ assures us that we shall not thereby change the language of M . If there are symbols in Σ not in Σ_1 , then none of these symbols appear in words of L . We may therefore introduce a “dead state” d into M with $\delta(d, a) = d$ for all a in Σ and $\delta(q, a) = d$ for all q in Q and a in $\Sigma - \Sigma_1$.

Now, to accept Σ^*-L , complement the final states of M . That is, let $M' = (Q, \Sigma_1, \delta, q_0, Q-F)$. Then M' accepts a word w if and only if $\delta(q_0, w)$ is in $Q-F$, that is, w is in Σ^*-L . Note that it is essential to the proof that M is deterministic and without ϵ moves.

Property 3: The regular sets are closed under intersection.

Proof: $L_1 \cap L_2 = (L_1' \cup L_2)'$, where the overbar denotes complementation with respect to an alphabet including the alphabets of L_1 and L_2 . Closure under intersection then follows from closure under union and complementation.

It is worth noting that a direct construction of a DFA for the intersection of two regular sets exists. The construction involves taking the Cartesian product of states, and we sketch the construction as follows:

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be two deterministic finite automata. Let

$$M = (Q_1 * Q_2, \Sigma, \delta, [q_1, q_2], F_1 * F_2)$$

where for all p_1 in Q_1 , p_2 in Q_2 , and a in Σ ,

$$\delta([p_1, p_2], a) = [\delta_1(p_1, a), \delta_2(p_2, a)]$$

Property 4: The class of regular sets is closed under substitution.

Proof: Let $R \in \Sigma^*$ be a regular set and for each a in Σ let $R_a \in \Delta^*$ be a regular set. Let $f: \Sigma \rightarrow \Delta^*$ be the substitutions denoting R and R_a . Replace each occurrence of the symbol a in the regular expression R by the regular expression for R_a . To prove that the resulting regular expression denotes $f(R)$, observe that the substitution of a union, product or closure is the union, product or closure of the substitution. [Thus for example, $f(L_1 L_2) = f(L_1) \cup f(L_2)$.] A simple induction on the number of operators in the regular expression completes the proof.

A type of substitution that is of special interest is the homomorphism. A *homomorphism* h is a substitution such that $h(a)$ contains a single string for each a . We generally take $h(a)$ to be the string itself, rather than the set containing that string. It is useful to define the *inverse homomorphic image* of a language L to be

$$h^{-1}(L) = \{\lambda \mid h(\lambda) \text{ is in } L\}$$

We also use, for string w ;

$$h^{-1}(w) = \{\lambda \mid h(\lambda) = w\}$$

Property 5: The class of regular sets is closed under homomorphism and inverse homomorphism.

Proof: Closure under homomorphism follows immediately from closure under substitution, since every homomorphism is a substitution, in which $h(a)$ has one member.

To show closure under inverse homomorphism, let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA accepting L , and let h be a homomorphism from Δ to Σ^* . We construct a DFA M' that accepts $h^{-1}(L)$ by reading symbol a in Δ and simulating M on $h(a)$. Formally, let $M' = (Q, \Sigma, \delta', q_0, F)$ and define $\delta'(q, a)$ for q in Q and a in Δ to be $\delta(q, h(a))$. Note that $h(a)$ may be long string, or ϵ , but δ is defined on all strings by extension. It is easy to show by induction on $|x|$ that $\delta'(q_0, x) = \delta(q, h(x))$. Therefore M' accepts x if and only if M accepts $h(x)$. That is, $L(M') = h^{-1}(L(M))$.

Property 6: The class of regular sets is closed under quotient with arbitrary sets.

Proof: Let $M = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton accepting some regular set R , and let L be an arbitrary language. The quotient R/L is accepted by a finite automaton $M' = (Q, \Sigma, \delta, q_0, F')$, which behaves like M except that the final states of M' are all states q of M such that $\delta(q_0, xy)$ is in F . Thus M' accepts R/L .

2. CONTEXT FREE GRAMMARS

Context-Free Grammars are a most powerful method of describing languages. Such grammars can describe certain features that have a recursive structure, which makes them useful in a variety of applications.

A grammar consists of a collection of substitution rules, also called productions. Each rule appears as a line in the grammar, comprising a symbol and a string separated by an arrow. The symbol is called a **variable**. The string consists of variables and other symbols called **terminals**. The variable symbols often are represented by capital letters. The terminals are analogous to the input alphabet and often are represented by lowercase letters, numbers, or special symbols. One variable is designed as the **start variable**. It usually occurs on the left-hand side of the topmost rule. For example, grammar G_1 contains three rules. G_1 's variables are A and B , where A is the start variable. Its terminals are 0 , 1 and $\#$.

You use a grammar to describe a language by generating each string of that language in the following manner.

1. Write down the start variable. It is the variable on the left-hand side of the top rule, unless specified otherwise.
2. Find a variable that is written down and a rule that starts with that variable. Replace the written down variable with the right-hand side of that rule.
3. Repeat step 2 until no variables remain.

For example, grammar G_1 generates the string 000#111. The sequence of substitutions to obtain a string is called a derivation. A derivation of string 000#111 in grammar G_1 is

$$A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$$

You may also represent the same information pictorially with a **parse tree**. An example of a parse tree is shown below

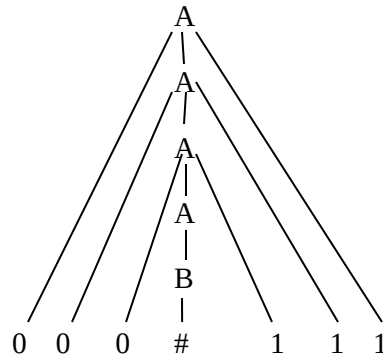


Fig: Parse tree for 000#111 in Grammar G_1

All strings generated in this way constitute the **language of the grammar**. We write $L(G_1)$ for the language of grammar G_1 . Some experimentation with the grammar G_1 shows us that $L(G_1)$ is $\{0^n\#1^n \mid n \geq 0\}$. Any language that can be generated by some context-free grammar is called a **context-free language** (CFL).

FORMAL DEFINITION OF A CONTEXT-FREE GRAMMAR

A **context-free grammar** is a 4-tuple (V, Σ, R, S) , where

1. V is a finite set called the **variables**,
2. Σ is a finite set, disjoint from V , called the **terminals**,
3. R is a finite set of **rules**, with each rule being a variable and a string of variables and terminals, and
4. $S \in V$ is the start variable.

3. DERIVATION TREES

The derivations in CFG can be represented using trees. Such trees representing derivations are called derivation trees.

A derivation tree for a CFG $G = (V, T, P, S)$ is a tree satisfying the following conditions:

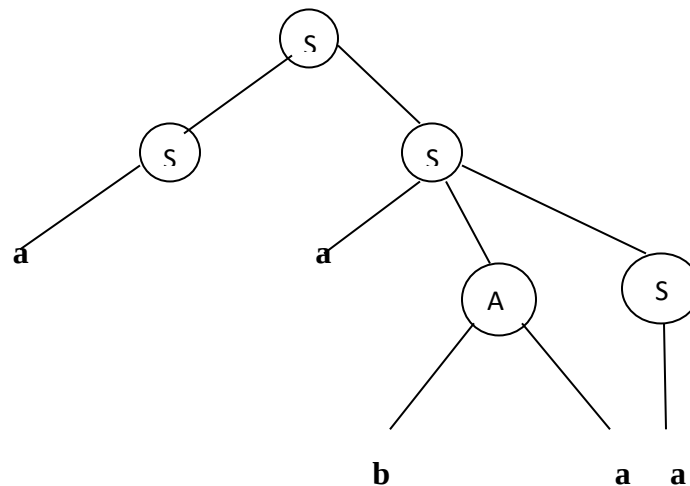
- (a) Every vertex has a label which is a variable or terminal or ϵ
- (b) The root has label S.
- (c) The label of an internal vertex is a variable.
- (d) If the vertices $n_1, n_2, \dots, n_k$ written with labels $x_1, x_2, \dots, x_k$ are the sons of vertex n with label A, then $A \rightarrow x_1 x_2 \dots x_k$ is a production in P.
- (e) A vertex n is a leaf if its label is $a \in \Sigma$ or ϵ .

For example:

Let $G = (\{S, A\}, \{a, b\}, P, S)$ where P consists of $S \rightarrow aAS/a/SS$

$A \rightarrow SbA/ba$

The derivation tree is given for the string aabaa as below:



2.1 LEFTMOST AND RIGHTMOST DERIVATION

Let G be a grammar $S \rightarrow 0B/1A$

$A \rightarrow 0/0S/1AA$

$B \rightarrow 1/1S/0BB$

For the string 00110101

Leftmost derivation:

$S \rightarrow 0\underline{B}$

$\rightarrow 00\underline{BB}$

$\rightarrow 001\underline{B}$

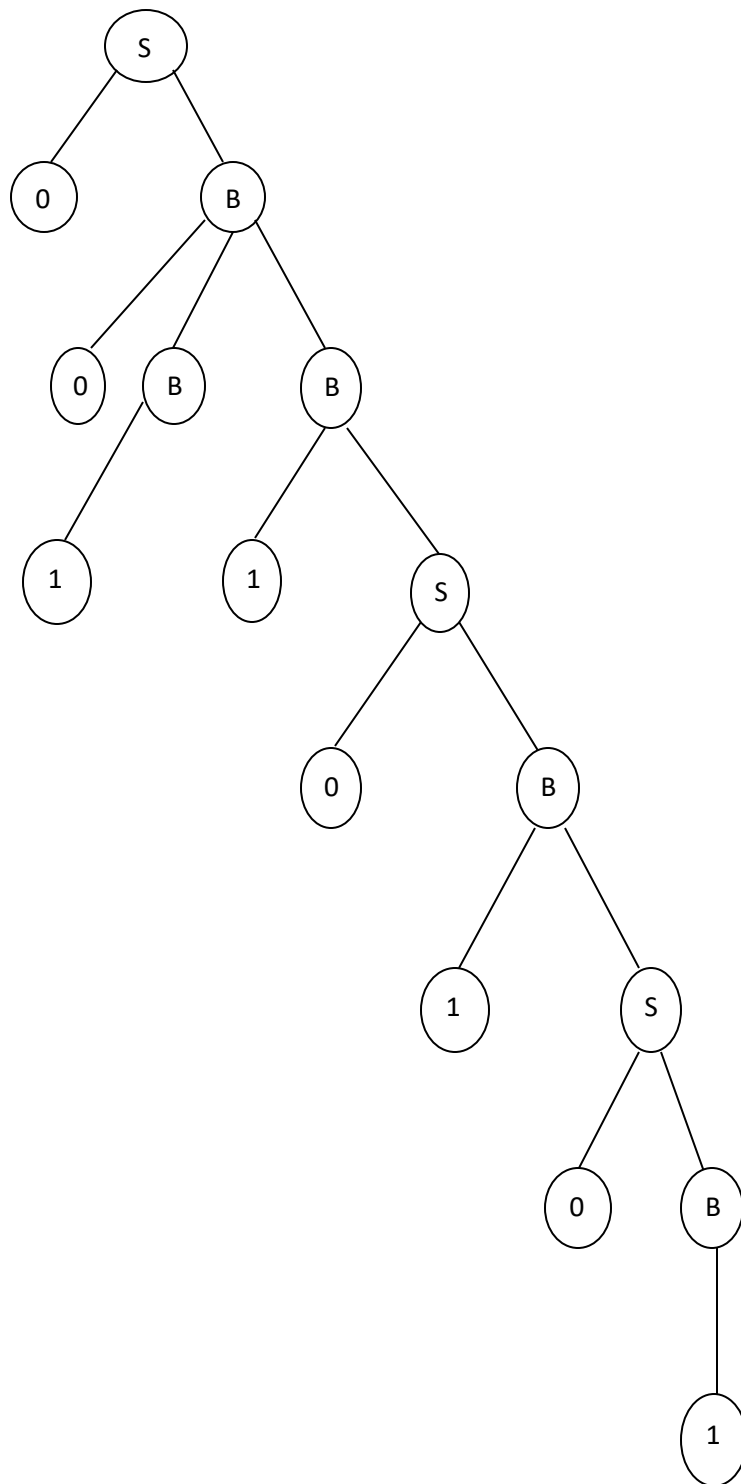
$\rightarrow 0011\underline{S}$

$\rightarrow 00110\underline{B}$

$\rightarrow 001101\underline{S}$

->0011010B
->00110101

Tree:



Rightmost derivation:

$S \rightarrow 0\underline{B}$

$\rightarrow 00\underline{BB}$

$\rightarrow 00\underline{B}1$

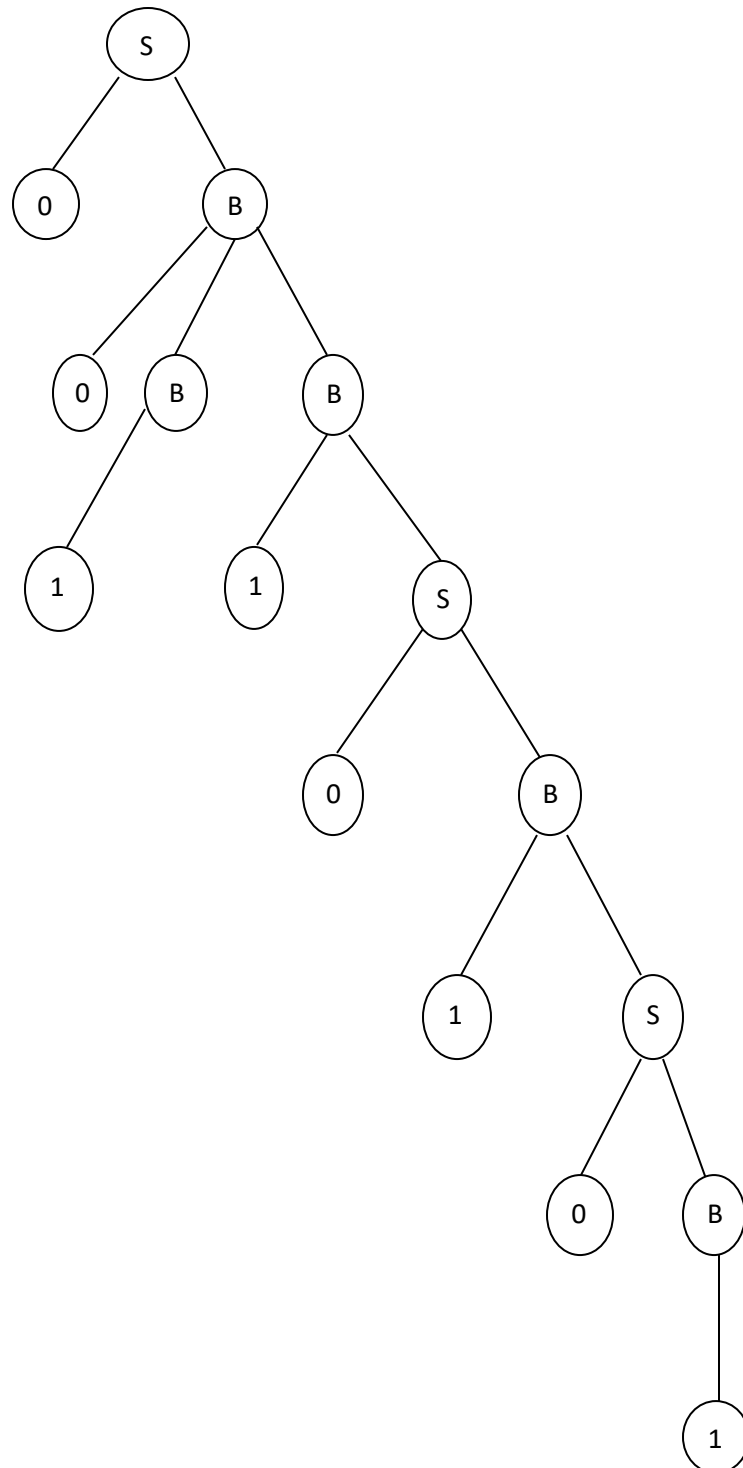
$\rightarrow 001\underline{S}1$

$\rightarrow 0011\underline{A}1$

$\rightarrow 00110\underline{S}1$

$\rightarrow 001101\underline{A}1$

$\rightarrow 00110101$



Example: Consider the following production

$S \rightarrow aB/bA$

$A \rightarrow aS/bAA/a$

$B \rightarrow bS/aBB/b$

for string aaabbabbba find LMD and RMD

LMD:

$S \rightarrow a\underline{B}$

$\rightarrow aa\underline{B}B$

$\rightarrow aaa\underline{B}BB$

$\rightarrow aaab\underline{B}B$

$\rightarrow aaabb\underline{B}$

$\rightarrow aaabba\underline{B}B$

$\rightarrow aaabbab\underline{B}$

$\rightarrow aaabbabb\underline{S}$

$\rightarrow aaabbabbb\underline{A}$

$\rightarrow aaabbabbba$

RMD:

$S \rightarrow a\underline{B}$

$\rightarrow aa\underline{B}B$

$\rightarrow aaBb\underline{S}$

$\rightarrow aaBbb\underline{A}$

$\rightarrow aa\underline{B}bba$

$\rightarrow aaa\underline{B}bba$

$\rightarrow aaa\underline{B}bbba$

$\rightarrow aaab\underline{S}bbba$

$\rightarrow aaabb\underline{A}bbba$

$\rightarrow aaabbabbba$

4. CHOMSKY NORMAL FORM (CNF)

A Context-Free Grammar 'G' is in Chomsky Normal Form if every production is of the form

- $A \rightarrow a$ (Non Terminal $\rightarrow$ Terminal)
- $A \rightarrow BC$ (Non Terminal $\rightarrow$ Non Terminal*Non Terminal)

is in G .

For Example:

Consider “ G ” whose productions are $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$. Then G is in Chomsky Normal Form.

Reduction to Chomsky Normal Form

The steps involved in reduction of CFG to CNF are

Step 1: Elimination of null productions and unit productions.

Let the resultant grammar be $G = (V_N, \Sigma, P, S)$

Step 2: Elimination of terminals on RHS.

We define $G = (V_N', \Sigma, P, S')$ where P and V_N' are constructed as follows:

(i) All the productions in P of the form $A \rightarrow a$ or $A \rightarrow BC$ are included in P_1 . All the variables in V_N are included in V_N' .

(ii) Consider $A \rightarrow X_1 X_2 \dots X_n$ with some terminal on RHS of X_i is a terminal say a_i , add a new variable C_{a_i} to V_N' and $C_{a_i} \rightarrow a_i$ to P_1 . In production $A \rightarrow X_1 X_2 \dots X_n$, every terminal on RHS is replaced by the corresponding new variables on the RHS are retained. The resulting production is added to P_1 . Thus we get $G_1 = (V_N', \Sigma, P_1, S)$.

Step 3: Restricting the number of variables on RHS.

For any production in P_1 , the RHS consists of either a single terminal or two or more variables. We define $G = (V_N', \Sigma, P_2, S)$ as follows:

(i) All productions in P_1 are added to P_2 if they are in the required form. All the variables in V_N'' .

(ii) Consider $A \rightarrow A_1 C_1$, $C_1 \rightarrow A_2 C_2$, $C_{m-2} \rightarrow A_{m-1} A_m$ and new variables are $C_1, C_2, \dots, C_{m-2}$.

Example 1:

Find the grammar in CNF equivalent to $S \rightarrow aAbB$, $A \rightarrow aA/a$, $B \rightarrow bB/b$

Step 1: As there are no unit productions or null productions, we need not carry out step 1. We proceed to step 2.

Step 2: Let $G_1 = (V_N', \{a, b\}, P, S)$ where P_1 and V_N' are constructed.

Add productions $A \rightarrow a$, $B \rightarrow b$ to P_1 .

Hence $S \rightarrow aAbB \sim (1)$

replaced as

a) $S \rightarrow C_a A C_b B$

Add new productions $C_a \rightarrow a$ and $C_b \rightarrow b$

b) $A \rightarrow aA$ becomes

$A \rightarrow C_a A$

c) $B \rightarrow bB$ becomes

$B \rightarrow C_b B$

Step 3:

a) $S \rightarrow C_a A C_b B$ is converted as

$S \rightarrow C_a D_1$ where

$D_1 \rightarrow A C_b B$

Again converted as $D_1 \rightarrow A D_2$

where $D_2 \rightarrow C_b B$

b) $A \rightarrow C_a A$

They are already in CNF

c) $B \rightarrow C_b B$

They are already in CNF

Hence the resultant CNF are

$S \rightarrow C_a D_1$

$D_1 \rightarrow A D_2$

$D_2 \rightarrow C_b B$

$A \rightarrow C_a A$

$B \rightarrow C_b B$

$A \rightarrow a$

$B \rightarrow b$

Example 2:

Find the CNF equivalent to the grammar $S \rightarrow \sim S/[S \cap S]/p/q$

where $\sim$, $[$, $\cap$, $]$, p , q are terminals.

Step 1: Consider the Grammar “G”.

$$S \rightarrow \sim S / [S \cap S] / p / q$$

Since there are no null productions and unit productions we can go for reduction.

Step 2: Consider the production (1),

$$S \rightarrow \sim S$$

Add new production $C_1 \rightarrow \sim$

The production $S \rightarrow \sim S$ becomes $S \rightarrow C_1 S$

Step 3: Consider the production (2),

$$S \rightarrow [S \cap S]$$

Add new productions $C_2 \rightarrow [$

$$C_3 \rightarrow \cap$$

$$C_4 \rightarrow]$$

$$\text{Now } S \rightarrow C_2 S C_3 S C_4$$

$$S \rightarrow C_2 D_1$$

$$\text{where } D_1 \rightarrow S C_3 S C_4$$

$$\text{and } D_2 \rightarrow C_3 S C_4$$

$$\text{So } D_1 \rightarrow S D_2$$

$$\text{Again } D_2 \rightarrow C_3 S C_4$$

$$\text{Let } D_3 \rightarrow S C_4$$

$$\text{And hence } D_2 \rightarrow C_3 D_3$$

The resultant CNF productions are

$$S \rightarrow C_1 S$$

$$C_1 \rightarrow \sim$$

$$S \rightarrow C_2 D_1$$

$$D_1 \rightarrow S D_2$$

$$D_2 \rightarrow C_3 D_3$$

$$D_3 \rightarrow S C_4$$

$$S \rightarrow p$$

$$S \rightarrow q$$

Hence derived.

Solved Problems:

Note: NT-Terminal T-Terminal

1. Convert the following grammar to Chomsky Normal Form(CNF)

G:

$S \rightarrow aAD$

$A \rightarrow aB/bAB$

$B \rightarrow b$

$D \rightarrow d$

Solution:

Consider $S \rightarrow aAD$

The production is of the form $NT \rightarrow T^*NT^*NT$

Replace a by C_a . We get, $S \rightarrow C_aAD$

Let $D_1 \rightarrow AD$. Then the production becomes $S \rightarrow C_a D_1$ which is in CNF.

Now consider $A \rightarrow aB$

This is of the form $NT \rightarrow T^*NT$

Since $C_a \rightarrow a$, $A \rightarrow aB$ becomes $A \rightarrow C_aB$ which is in CNF.

Let us now consider $A \rightarrow bAB$.

This production is of the form $NT \rightarrow T^*NT^*NT$

Replace b by C_b and AB by D_2 . We get, $A \rightarrow C_b D_2$.

The other two productions $B \rightarrow b$ and $D \rightarrow d$ are already in CNF.

Hence the resultant CNF productions are

$S \rightarrow C_a D_1$

$D_1 \rightarrow AD$

$A \rightarrow C_a B$

$A \rightarrow C_b D_2$

$D_2 \rightarrow AB$

$B \rightarrow b$

$D \rightarrow d$

2. Convert the following grammar to Chomsky Normal Form (CNF)

G:

$S \rightarrow abSb/a/aAb$

$A \rightarrow bS/aAAb$

Solution:

Consider $S \rightarrow abSb$

This production is of the form $NT \rightarrow T^*NT^*T^*NT$

Replacing a by C_a and b by C_b we get, $S \rightarrow C_a C_b S C_b$

This is of the form $NT \rightarrow NT^*NT^*NT^*NT$.

So we replace $C_b S C_b$ by D_1 . Hence the production becomes, $S \rightarrow C_a D_1$ which is of the CNF.

(Replace $S C_b$ by D to convert D_1 into CNF. So $D \rightarrow S C_b$)

Since $S \rightarrow a$ is already in CNF we now consider $S \rightarrow aAB$

The above production is of the form $NT \rightarrow T^*NT^*T$. Replace a by C_a and b by C_b .

So now, $S \rightarrow C_a A C_b$. Replace $A C_b$ by D_2 to convert the production to CNF.

The production now becomes $S \rightarrow C_a D_2$.

Let us now consider the production $A \rightarrow bS$. Replacing b by C_b the grammar gets converted into CNF, $A \rightarrow C_b S$.

The last production to be converted to CNF is $A \rightarrow aAAb$.

Replacing C_a and b by C_b we get $A \rightarrow C_a A A C_b$. Let us now consider $D_3 \rightarrow A D_2$ where

$D_2 \rightarrow A C_b$. So the production becomes $A \rightarrow C_a D_3$ which is in CNF.

Hence the resultant CNF productions are

$D \rightarrow SC_b$

$D_1 \rightarrow C_b D$

$S \rightarrow C_a D_1$

$D_2 \rightarrow AC_b$

$S \rightarrow C_a D_2$

$A \rightarrow C_a D_3$

$D_3 \rightarrow AD_2$

$D_2 \rightarrow AC_b$

$S \rightarrow a$

3. Convert the following grammar to Chomsky Normal Form (CNF)

G:

$S \rightarrow ASA/bA$

$A \rightarrow B/S$

$B \rightarrow C$

Solution:

Consider $S \rightarrow ASA$.

The above production is of the form $NT \rightarrow NT*NT*NT$

Let $D_1 \rightarrow SA$. Replacing SA the production becomes $S \rightarrow AD_1$ which is in CNF.

Now consider $S \rightarrow bA$. Replace b by C_b in order to convert the production to CNF.

$S \rightarrow bA$ becomes $S \rightarrow C_b A$.

Consider the productions $A \rightarrow B$, $A \rightarrow S$ and $B \rightarrow C$. They can also be written as $A \rightarrow \epsilon B$, $A \rightarrow \epsilon S$, $B \rightarrow \epsilon C$. If we assume $C_a \rightarrow \epsilon$ then the productions become $A \rightarrow C_a B$, $A \rightarrow C_a S$, $B \rightarrow C_a C$ which are in CNF.

Hence the resultant CNF productions are

$S \rightarrow AD_1$

$D_1 \rightarrow SA$

$A \rightarrow C_a B$

$S \rightarrow C_b A$

$A \rightarrow C_a S$

$B \rightarrow C_a C$

4. Convert the following grammar to Chomsky Normal Form (CNF)

G:

$S \rightarrow 1A/0B$

$A \rightarrow 1AA/0S/0$

$B \rightarrow 0BB/1S/1$

Solution:

Consider $S \rightarrow 1A$. The above production is of the form $NT \rightarrow T^*NT$. In order to convert it to CNF, we replace 1 by C_a . Upon replacing we get, $S \rightarrow C_a A$ which is in CNF.

Now consider $S \rightarrow 0B$. The above production is of the form $NT \rightarrow T^*NT$. In order to convert it to CNF, we replace 0 by C_b . Upon replacing we get, $S \rightarrow C_b B$ which is in CNF.

The next production to be considered is $A \rightarrow 1AA$ which is of the form $NT \rightarrow T^*NT^*NT$. Let us assume that $D_1 \rightarrow AA$. Also we know $C_a \rightarrow 1$. The production becomes $A \rightarrow C_a D_1$ which is in CNF.

The next production $A \rightarrow 0S$ is of the form $NT \rightarrow T^*NT$. We know $C_b \rightarrow 0$. Upon replacing we get, $A \rightarrow C_b S$ which is in CNF. The other production $A \rightarrow 0$ is already in CNF.

Now take into account the production $B \rightarrow 0BB$ which is of the form $NT \rightarrow T^*NT^*NT$. Let us assume that $D_2 \rightarrow BB$. Also we know $C_b \rightarrow 0$. The production becomes $B \rightarrow C_b D_2$ which is in CNF.

The next production $B \rightarrow 1S$ is of the form $NT \rightarrow T^*NT$. We know $C_a \rightarrow 1$. Upon replacing we get, $B \rightarrow C_a S$ which is in CNF. The other production $B \rightarrow 1$ is already in CNF.

Hence the resultant CNF productions are

$S \rightarrow C_a A$

$S \rightarrow C_b B$

$D_1 \rightarrow AA$

$A \rightarrow C_a D_1$

$A \rightarrow C_b S$

$A \rightarrow \epsilon$

$B \rightarrow C_a S$

$B \rightarrow \epsilon$

5. GREIBACH NORMAL FORM (GNF)

A Context-Free Grammar 'G' is in Greibach Normal Form if every production is of the form

- $A \rightarrow a\alpha$ where $\alpha \in V_N^*$ and $a \in \Sigma$
(Non Terminal $\rightarrow$ Terminal * any number of Non Terminals)
- $A \rightarrow a$ (Non Terminal $\rightarrow$ Terminal)

For Example:

$S \rightarrow aAB$

$A \rightarrow bC$

$B \rightarrow b$

$C \rightarrow c$

are in GNF.

Greibach Normal Form Algorithm:

```
begin
  for  $k:=1$  to  $m$  do
    begin
      for  $j=1$  to  $k-1$  do
        for each production of the form  $A_k \rightarrow A_j \alpha$  do
          begin
            for all productions  $A_j \rightarrow \beta$  do
              add production  $A_k \rightarrow \beta \alpha$ ;
              remove production  $A_k \rightarrow A_j \alpha$ 
            end;
          for each production of the form  $A_k \rightarrow A_k \alpha$  do
            begin
              add production  $B_k \rightarrow \alpha$  and  $B_k \rightarrow \alpha B_k$ ;
              remove production  $A_k \rightarrow A_k \alpha$ 
            end;
          for each production  $A_k \rightarrow \beta$ , where  $\beta$  does
            not begin with  $A_k$  do
              add production  $A_k \rightarrow \beta B_k$ 
            end
          end
    end
  end
```

Reduction to Greibach Normal Form

Lemma (1):

Let $G = (V, \Sigma, P, S)$ be a CFG. Let $A \rightarrow B\gamma$ be an A-production in P . Let the B-production be $B \rightarrow B_1|B_2|\dots|B_n$

Lemma (2):

Let $G = (V, \Sigma, P, S)$ be a CFG. Let $A \rightarrow B\gamma$ be an A-production be

$$A \rightarrow A\alpha_1|\dots A\alpha_r|B_1|\dots|B_n$$

then z be a new variable when P_1 is defined as:

(i) The set of A-productions in P_1 are

$$A \rightarrow \beta_1|\beta_2|\dots|\beta_n$$

$$A \rightarrow \beta_1z|\beta_2z|\dots|\beta_nz$$

(ii) The set of z-productions in P_1 are

$$z \rightarrow \alpha_1|\alpha_2|\dots|\alpha_n$$

$$z \rightarrow \alpha_1z|\alpha_2z|\dots|\alpha_nz$$

Example:

Construct Equivalent GNF for the CFG $S \rightarrow AA/a, A \rightarrow SS/b$

Step 1: The given grammar is in CNF. S and A are renamed as A_1 and A_2 .

Hence the productions become $A_1 \rightarrow A_2 A_2/a, A_2 \rightarrow A_1 A_1/b$

There is no need for null production or unit production elimination because the production is already in CNF.

Step 2: The A_1 productions are in the required form. They are $A_1 \rightarrow A_2 A_2/a$.

The production $A_2 \rightarrow b$ is also in the required form whereas we have to apply lemma 1 to convert the production $A_2 \rightarrow A_1 A_1$ to the required form. Applying lemma 1 we get,

$$A_2 \rightarrow A_2 A_2 A_1 \quad A_2 \rightarrow aA_1$$

Step 3: We apply lemma 2 to A_2 productions as we have $A_2 \rightarrow A_2 A_2 A_1$

Let z_2 be a new variable. The resulting productions are

$$A_2 \rightarrow aA_1$$

$$A_2 \rightarrow b$$

$$A_2 \rightarrow aA_1z_2$$

$$A_2 \rightarrow bz_2$$

$$z_2 \rightarrow A_2 A_1$$

$$z_2 \rightarrow A_2 A_1 z_2$$

Step 4: (i) The A_2 productions are $A_2 \rightarrow aA_1 / b / aA_1z_2$

(ii) Among the A_1 productions we retain $A_1 \rightarrow a$ and eliminate $A_1 \rightarrow A_2 A_2$ using lemma 1.

The resulting productions are $A_1 \rightarrow aA_1 A_2 / bA_2$, $A_1 \rightarrow aA_1z A_2 / bz_2A_2$.

The set of all A_1 -productions is $A_1 \rightarrow a / aA_1 A_2 / bA_2 / aA_1z A_2 / bz_2A_2$

Step 5: The z_2 productions to be modified are $z_2 \rightarrow A_2 A_1$, $z_2 \rightarrow A_2 A_1 z_2$.

Applying lemma 1 we get $z_2 \rightarrow aA_1A_1 / bA_1 / aA_1z_2 A_1 / bz_2A_1$

$z_2 \rightarrow aA_1A_1z_2 / bA_1 z_2 / aA_1z_2 A_1 z_2 / bz_2A_1 z_2$

Hence the equivalent grammar is

$A_1 \rightarrow a / aA_1 A_2 / bA_2 / aA_1z A_2 / bz_2A_2$

$A_2 \rightarrow aA_1 / b / aA_1z_2 / bz_2$

$z_2 \rightarrow aA_1A_1 / bA_1 / aA_1z_2 A_1 / bz_2A_1 z_2$

$z_2 \rightarrow aA_1A_1z_2 / bA_1 z_2 / aA_1z_2 A_1 z_2 / bz_2A_1 z_2$

Solved Problems:

Note: NT-Terminal T-Terminal

1. Let us convert to Greibach normal form the grammar

$G = (\{A_1, A_2, A_3\}, \{a, b\}, P, A_1)$, where P consists of the following:

$A_1 \rightarrow A_2A_3$

$A_2 \rightarrow A_3A_1 / b$

$A_3 \rightarrow A_1A_2 / a$

Solution:

Step 1: Since the right-hand side of the productions for A_1 and A_2 start with terminals or highest-numbered variables, we begin with the production $A_3 \rightarrow A_2 A_3$ is the only production with A_1 on the left.

The resultant set of productions is: $A_1 \rightarrow A_2 A_3$

$$A_2 \rightarrow A_3 A_1 / b$$

$$A_3 \rightarrow A_2 A_3 A_2 / a$$

Since the right side of the production $A_3 \rightarrow A_2 A_3 A_2$ begins with a lower-numbered variable, we substitute for the first occurrence of A_2 both $A_3 A_1$ and b . Thus $A_3 \rightarrow A_2 A_3 A_2$ is replaced by $A_3 \rightarrow A_3 A_1 A_3 A_2$ and $A_3 \rightarrow b A_3 A_2$. The new set is

$$A_1 \rightarrow A_2 A_3$$

$$A_2 \rightarrow A_3 A_1 / b$$

$$A_3 \rightarrow A_3 A_1 A_3 A_2 / b A_3 A_2 / a$$

We now apply lemma 2 to the productions

$$A_3 \rightarrow A_3 A_1 A_3 A_2 / b A_3 A_2 / a$$

Let z_2 be a new variable. The resulting productions are

$$A_1 \rightarrow A_2 A_3$$

$$A_2 \rightarrow A_3 A_1 / b$$

$$A_3 \rightarrow b A_3 A_2 z_2 / a z_2 / b A_3 A_2 / a$$

$$z_2 \rightarrow A_1 A_3 A_2 / A_1 A_3 A_2 z_2$$

Step 2: Now all the productions with A_3 on the left have right-hand sides that start with terminals. These are used to replace A_3 in the productions $A_2 \rightarrow A_3 A_1$ and then the productions with A_2 on the left are used to replace A_2 in the production $A_1 \rightarrow A_2 A_3$. The result is the following.

$$A_3 \rightarrow b A_3 A_2 z_2$$

$$A_3 \rightarrow b A_3 A_2$$

$$A_3 \rightarrow a z_2$$

$$A_3 \rightarrow a$$

$$A_2 \rightarrow b A_3 A_2 z_2 A_1$$

$$A_2 \rightarrow b A_3 A_2 A_1$$

$$A_2 \rightarrow a z_2 A_1$$

$$A_2 \rightarrow a A_1$$

$$A_2 \rightarrow b$$

$$A_1 \rightarrow bA_3A_2z_2A_1A_3$$

$$A_1 \rightarrow b z_2A_1A_3$$

$$A_1 \rightarrow bA_3$$

$$z_2 \rightarrow A_1A_3A_2$$

$$A_1 \rightarrow bA_3A_2A_1A_3$$

$$A_1 \rightarrow b A_1A_3$$

$$z_2 \rightarrow A_1A_3A_2z_2$$

Step 3: The two z_2 productions are converted to proper form, resulting in 10 more productions. That is, the productions

$$z_2 \rightarrow A_1A_3A_2$$

$$z_2 \rightarrow A_1A_3A_2z_2$$

are altered by substituting the right side of each of the five productions with A_1 on the left for the first occurrences of A_1 . Thus $z_2 \rightarrow A_1A_3A_2$ becomes

$$z_2 \rightarrow bA_3A_2z_2A_1A_3 A_3A_2$$

$$z_2 \rightarrow az_2A_1A_3A_3A_2$$

$$z_2 \rightarrow bA_3A_3A_2$$

$$z_2 \rightarrow bA_3A_2z_2A_1A_3 A_3A_2$$

$$z_2 \rightarrow aA_1A_3A_3A_2$$

The other production for z_2 is replaced similarly. The final set of productions is

$$A_3 \rightarrow bA_3A_2z_2$$

$$A_3 \rightarrow bA_3A_2$$

$$A_3 \rightarrow az_2$$

$$A_3 \rightarrow a$$

$$A_2 \rightarrow bA_3A_2z_2A_1$$

$$A_2 \rightarrow bA_3A_2A_1$$

$$A_2 \rightarrow az_2A_1$$

$$A_2 \rightarrow aA_1$$

$$A_2 \rightarrow b$$

$$A_1 \rightarrow bA_3A_2z_2A_1A_3$$

$$A_1 \rightarrow bA_3A_2A_1A_3$$

$$A_1 \rightarrow b z_2A_1A_3$$

$$A_1 \rightarrow b A_1A_3$$

$$A_1 \rightarrow bA_3$$

$$z_2 \rightarrow bA_3A_2z_2A_1A_3 A_3A_2$$

$$z_2 \rightarrow bA_3A_2z_2A_1A_3 A_3A_2$$

$$z_2 \rightarrow az_2A_1A_3A_3A_2z_2$$

$$z_2 \rightarrow az_2A_1A_3A_3A_2$$

$$z_2 \rightarrow bA_3A_3A_2z_2$$

$$z_2 \rightarrow bA_3A_3A_2$$

$$z_2 \rightarrow bA_3A_2A_1A_3 A_3A_2z_2$$

$$z_2 \rightarrow bA_3A_2A_1A_3 A_3A_2$$

$$z_2 \rightarrow aA_1A_3A_3A_2z_2$$

$$z_2 \rightarrow aA_1A_3A_3A_2$$

5. AMBIGUOUS AND UNAMBIGUOUS GRAMMARS

Ambiguous Grammar: The production should be in GNF and if the production contains more than one derivation or derivation tree then it is termed as an ambiguous grammar.

For example consider the production, $S \rightarrow S+S/S*S/a/b$

We have to derive the string $a+a*b$

The derivation is as follows:

$S \rightarrow S+S$
 $\rightarrow S+S*S$
 $\rightarrow a+a*b$

Hence derived.

The same string can also be derived in the following manner

$S \rightarrow S*S$
 $\rightarrow S+S*S$
 $\rightarrow a+a*b$

Hence from the above two derivations it is obvious that the string can be derived from more than one derivation. Therefore it is known as ambiguous grammar.

Unambiguous Grammar: If the production contains exactly one derivation or one derivation tree then it is termed as an unambiguous grammar.

Consider an example:

$S \rightarrow a/abSb/aAb$
 $A \rightarrow bS/aAAb$, the word to be derived is $W=abab$

Derivation:

$S \rightarrow abSb$
 $\rightarrow abab$

Hence derived.

Now consider

$S \rightarrow aAb$
 $\rightarrow abSb$
 $\rightarrow ababSbb$
 $\rightarrow ababbbb$

$S \rightarrow abSb$

$\rightarrow ababSbb$

$\rightarrow ababbbb$

$A \rightarrow aAAb$

$\rightarrow abSbSb$

$\rightarrow ababab$

$A \rightarrow bS$

$\rightarrow ba$

From the above derivations it is obvious that the string abab can have only one derivation. Hence the grammar can be termed as an unambiguous grammar.

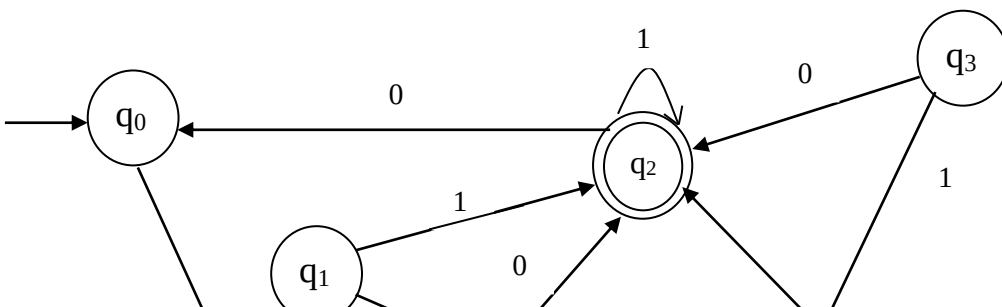
6. THE MYHILL-NERODE THEOREM

Begin

```
    for  $p$  in  $F$  and  $q$  in  $Q-F$  do mark  $(p,q)$ ;  
    for each pair of distinct states  $(p,q)$            in  $F^*F$   
or  $(Q-F)^*(Q-F)$  do  
    if for some input  $a, (\delta(p,a), \delta(q,a))$  is  
    marked then  
    begin  
        Mark( $p,q$ )  
    recursively mark all unmarked pairs on the  
    list for  $(p,q)$  and on the lists of the other  
    pairs that are marked at this step.  
    end  
    else  
    for all input symbols  $a$  do  
        Put( $p,q$ ) on the list for  $(\delta(p,a), \delta(q,a))$   
        unless  $\delta(p,a) = \delta(q,a)$   
end
```

Construction of Minimum Automaton:

Consider the following transition diagram



↗ 0

Now construct the transition table:

δ	0	1
q ₀	q ₁	q ₅
q ₁	q ₆	q ₂
q ₂	q ₀	q ₂
q ₃	q ₂	q ₆
q ₄	q ₇	q ₅
q ₅	q ₂	q ₆
q ₆	q ₆	q ₄
q ₇	q ₆	q ₂

THE MYHILL-NERODE THEOREM

```
Begin
    for p in F and q in Q-F do mark (p,q);
    for each pair of distinct states (p,q)           in F*F
or (Q-F)*(Q-F) do
    if for some input a, (δ(p,a), δ(q,a)) is
marked then
    begin
        Mark(p,q)
        recursively mark all unmarked pairs on the
        list for (p,q) and on the lists of the other
        pairs that are marked at this step.
    end
    else
    for all input symbols a do
        Put(p,q) on the list for , (δ(p,a), δ(q,a))
        unless δ(p,a) = δ(q,a)
    end
```

Step 1: Construction of π_0 , by def 0-equivalence

$\Pi_0 = \{Q_1^0, Q_2^0\}$ where

Q_1^0 is the set of final states and

$$Q_2^0 = Q - Q_1^0$$

hence

$$Q_1^0 = F = \{q_2\}$$

$$Q_2^0 = Q - Q_1^0$$

So

$$\Pi_0 = \{\{q_2\}, \{q_0, q_1, q_3, q_4, q_5, q_6, q_7\}\}$$

The $\{q_2\}$ in Π_0 cannot be partitioned.

$$\text{So } Q_1' = \{q_2\}$$

Consider q_0 and $q_1 \in Q_2^0$.

The entries under the 0 – column corresponding to q_0 and q_1 are q_1 and q_6 ; they lie in Q_2^0 . The entries in 1- column are q_5 and q_2 , $q_2 \in Q_1^0$ and $q_5 \in Q_2^0$. Therefore q_0 and q_1 are not 1-Equivalent. Similarly q_0 is 1 – Equivalent to q_3, q_5 and q_7 .

Now consider q_0 and q_4 . The entries under the 0-column are q_1 and q_7 both are in Q_2^0 . The entries under 1-column are q_5 , so q_4 and q_0 are 1- Equivalent.

Similarly q_0 is 1- Equivalent to q_6 ,

$$\text{So, } Q_2' = \{q_0, q_4, q_6\}$$

Repeat the construction by considering q_1 as one of the states q_3, q_5, q_7 . Now q_1 is not 1 – Equivalent to q_3 and q_5 but 1 – Equivalent to q_7 hence $Q_3' = \{q_1, q_7\}$. The left over elements in Q_2^0 are q_3 and q_5 .

Therefore,

$$\Pi_1 = \{\{q_2\}, \{q_0, q_4, q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$$

The $\{q_2\}$ is also in Π_2 as it cannot be further partitioned. Now the entries under 0 – column to q_0 and q_4 are q_1 and q_7 and these lie in the same equivalence class in Π_1 . The entries under 1- column are q_5 . So q_0 and q_4 are 2- Equivalent. But q_0 and q_6 are not 2-Equivalent. Hence $\{q_0, q_4, q_6\}$ is partitioned into $\{q_0, q_4\}$ and $\{q_6\}$. q_1 and q_7 are 2-Equivalent.

$$\Pi_2 = \{\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$$

q_0 and q_4 are 3 – Equivalent

Therefore

$$\Pi_3 = \{\{q_2\}, \{q_0, q_4\}, \{q_6\}, \{q_1, q_7\}, \{q_3, q_5\}\}$$

The minimum state automaton is

$$M' = (Q', \{0, 1\}, \delta', q_0', F')$$

where

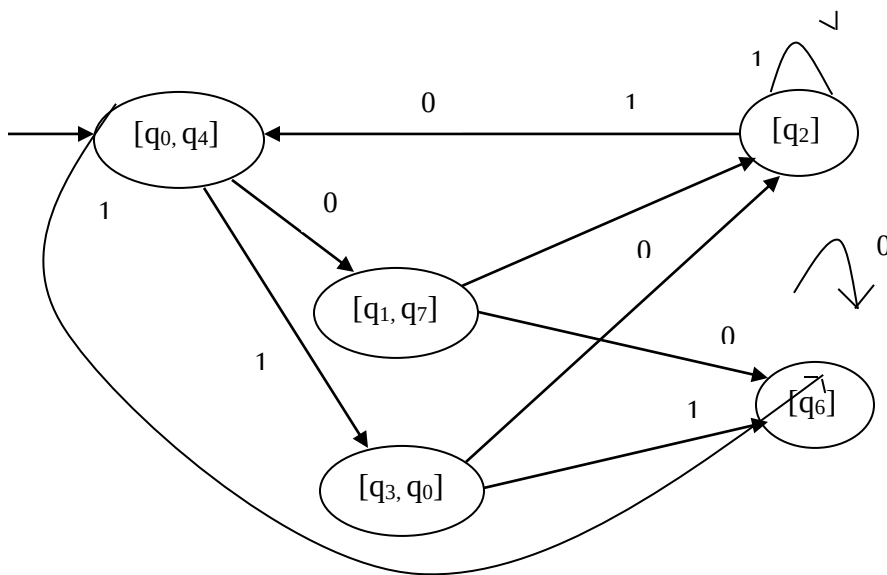
$$Q' = \{[q_2], [q_0, q_4], [q_6], [q_1, q_7], [q_3, q_5]\}$$

$$q_0' = [q_0, q_4]$$

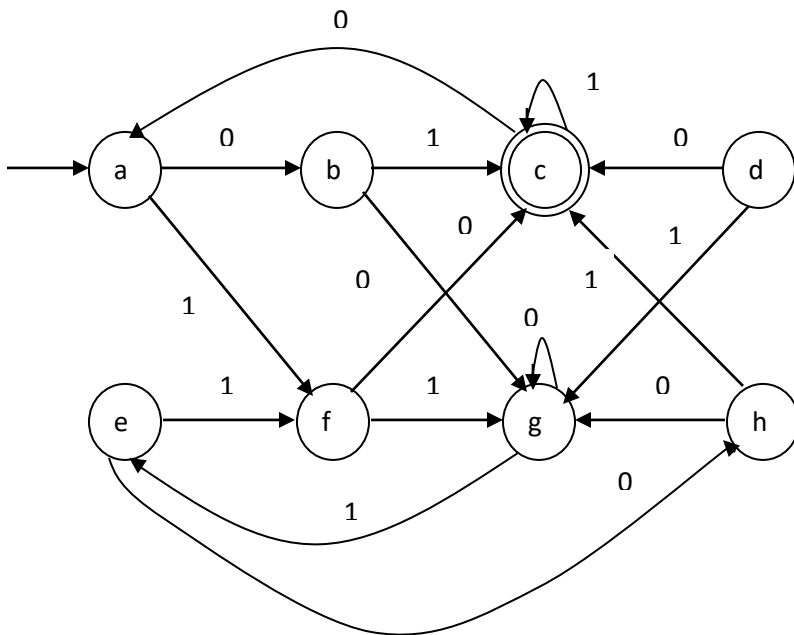
$$F' = [q_2]$$

and δ' is defined by

δ	0	1
$[q_0, q_4]$	$[q_1, q_7]$	$[q_3, q_5]$
$[q_1, q_7]$	$[q_6]$	$[q_2]$
$[q_2]$	$[q_0, q_4]$	$[q_2]$
$[q_3, q_5]$	$[q_2]$	$[q_6]$
$[q_6]$	$[q_6]$	$[q_0, q_4]$



Example 2: Minimize the finite automata given below



The transition table can be given as

δ	0	1
a	b	f
b	g	c
c	f	c
d	c	g
e	h	f
f	c	g
g	g	e
h	g	c

$$\Pi_0 = \{\{c\}, \{a, b, d, e, f, g, h\}\}$$

Where class A = {c} and Class B = {a, b, d, e, f, g, h}

Class B:

Case (i) : comparing a & b

$$\delta(a,0) = b \in B$$

$$\delta(b,0) = g \in B$$

$$\delta(a,1) = f \in B$$

$$\delta(b,1) = c \in A$$

So b can be partitioned from a

Case (ii) : comparing a & d

$$\delta(a,0) = b \in B$$

$$\delta(d,0) = c \in A$$

$$\delta(a,1) = f \in B$$

$$\delta(d,1) = g \in B$$

So d can be partitioned from a

Case (iii): comparing a & e

$$\delta(a,0) = b \in B$$

$$\delta(e,0) = h \in B$$

$$\delta(a,1) = f \in B$$

$$\delta(e,1) = f \in B$$

So e can't be partitioned from a

Case (iv) : comparing a & f

$$\delta(a,0) = b \in B$$

$$\delta(f,0) = c \in A$$

$$\delta(a,1) = f \in B$$

$$\delta(f,1) = g \in B$$

So f can be partitioned from a

Case (v): comparing a & g

$$\delta(a,0) = b \in B$$

$$\delta(g,0) = g \in B$$

$$\delta(a,1) = f \in B$$

$$\delta(g,1) = e \in B$$

So g can't be partitioned from a

Case (vi): comparing a & h

$$\delta(a,0) = b \in B$$

$$\delta(h,0) = g \in B$$

$$\delta(a,1) = f \in B$$

$$\delta(h,1) = c \in A$$

So h can be partitioned from a

$$\Pi_1 = \{\{c\}, \{a, e, g\}, \{b, d, f, h\}\}$$

Class B:

Case (i): comparing a & e

$$\delta(a,0) = b \in C$$

$$\delta(e,0) = h \in C$$

$$\delta(a,1) = f \in C$$

$$\delta(e,1) = f \in C$$

So e can't be partitioned from a

Case (i): comparing a & g

$$\delta(a,0) = b \in C$$

$$\delta(g,0) = g \in B$$

$$\delta(a,1) = f \in C$$

$$\delta(g,1) = e \in B$$

So g can't be partitioned from a

Class C

Case (i): comparing b & d

$$\delta(b,0) = g \in B$$

$$\delta(d,0) = c \in A$$

$$\delta(b,1) = c \in A$$

$$\delta(d,1) = g \in B$$

So d can be partitioned from b.

Case (i): comparing b & f

$$\delta(b,0) = g \in B$$

$$\delta(f,0) = c \in A$$

$$\delta(b,1) = c \in A$$

$$\delta(f,1) = g \in B$$

So f can be partitioned from b.

Case (iii): comparing b & h

$$\delta(b,0) = g \in C$$

$$\delta(h,0) = g \in C$$

$$\delta(b,1) = c \in C$$

$$\delta(h,1) = c \in C$$

So h can't be partitioned from b.

$$\Pi_2 = \{\{c\}, \{a, e\}, \{g\} \{b, h\}, \{d, f\}\}$$

